Transformer-Based Universal Source Coding

Dan Vu

2025-03-01

1 The Question

Universal source coders don't know the true distribution of the data they're compressing. They have to figure it out on the fly. The classic result here is the Laplace estimator – for binary IID sequences it maintains a running count of 0s and 1s and estimates p as:

$$\hat{p}_1 = \frac{k+1}{n+2}$$

where k is ones seen so far out of n total bits. It starts from a uniform prior and converges to the true p as the sequence grows. The redundancy – the extra bits per symbol paid over the entropy lower bound – decays as $O(1/n)$.

A natural question: if you train a model on data from a known source beforehand, can you skip that warm-up cost and start closer to entropy from bit one? And if so, how much training do you need before it's worth it?

This is the question Host-Madsen (2021) addresses theoretically for binary IID sources and Markov chains. The answer is surprisingly optimistic: a learned coder only needs about $m \sim \frac{l}{\log l}$ training samples to beat a universal coder on sequences of length l. You need fewer training samples than the length of the sequences you want to encode.

This project is an empirical look at that claim. We train a small GPT (minGPT) on binary sequences and benchmark it against the Laplace estimator across two conditions that test when offline learning actually helps.

2 Setup

Both coders are used as sequential probability models. At each position in a sequence, the model outputs a probability for the next bit. We compute bits per symbol directly from those probabilities:

$$\text{BPS} = \frac{1}{N}\sum_{i=1}^{N} -\log_2 \hat{p}(x_i \mid x_1^{i-1})$$

Laplace estimates $\hat{p}$ from the running count. The transformer takes the preceding bits as context and outputs a probability from its learned weights. Same formula, same comparison baseline.

Two experimental conditions:

- **IID, unknown p**: each test sequence is drawn with a fresh $p \sim \text{Uniform}(0, 1)$. The transformer is trained on sequences from this same distribution, so it has seen many different p values but never knows the current one ahead of time.
- **Fixed p**: the transformer is trained exclusively on sequences from a single fixed p. It has the chance to learn the true distribution offline. Laplace doesn't get this since it always starts fresh regardless of how much data you've generated at training time.

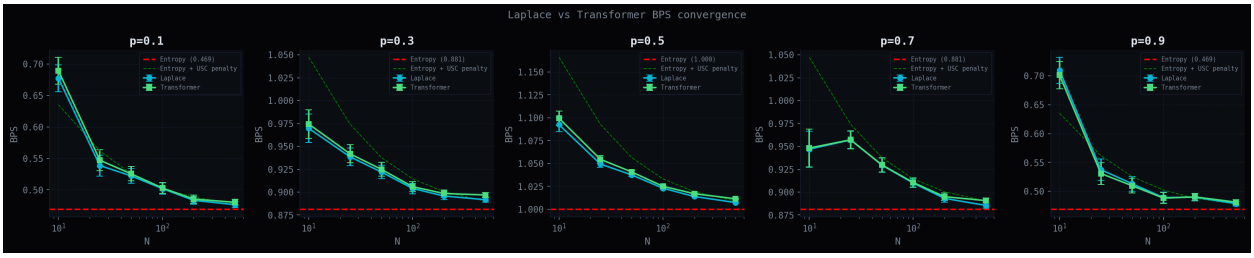
The difference between these two conditions is exactly the question: does offline learning help, and when?

3 IID – Unknown p

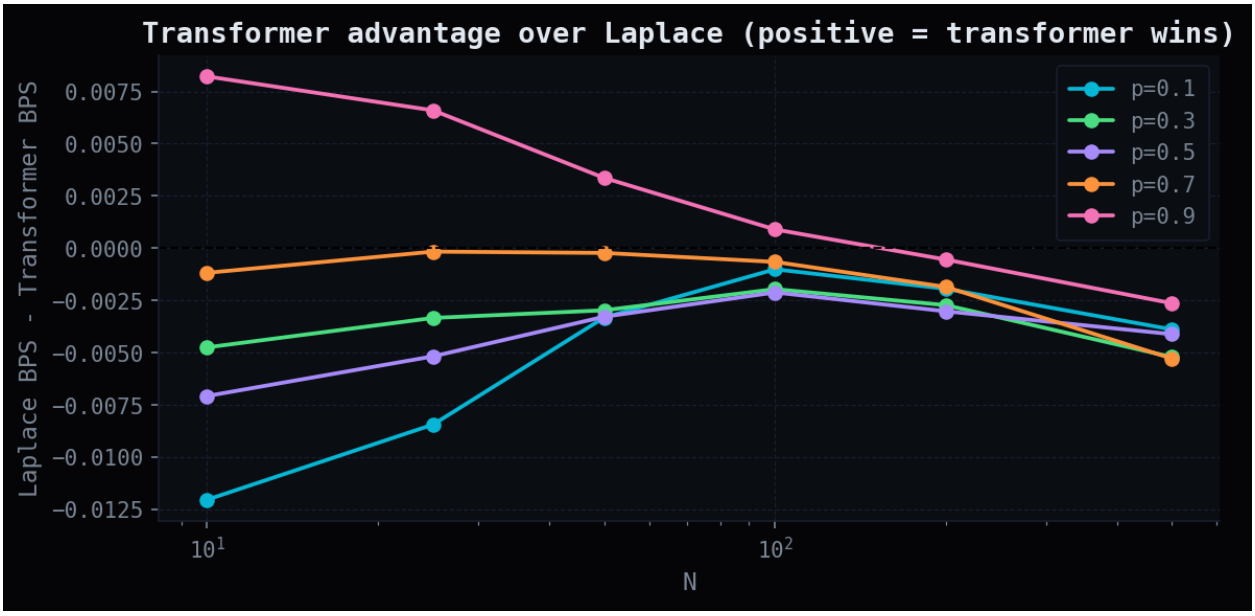
First the IID case, where p changes every sequence.

```
import sys; sys.path.insert(0, '..')
from assets.matplotlib_theme import apply_void_theme

# BPS vs sequence length N across p ∈ {0.1, 0.3, 0.5, 0.7, 0.9}: Laplace and Transformer
```




```
# Gap (Laplace BPS − Transformer BPS) vs N for IID unknown-p, one line per p value.
```



The transformer tracks Laplace almost exactly across all tested p values. The gap plot is noisy around zero, demonstrating no consistent advantage in either direction, and what small differences exist are within the variance of the experiment.

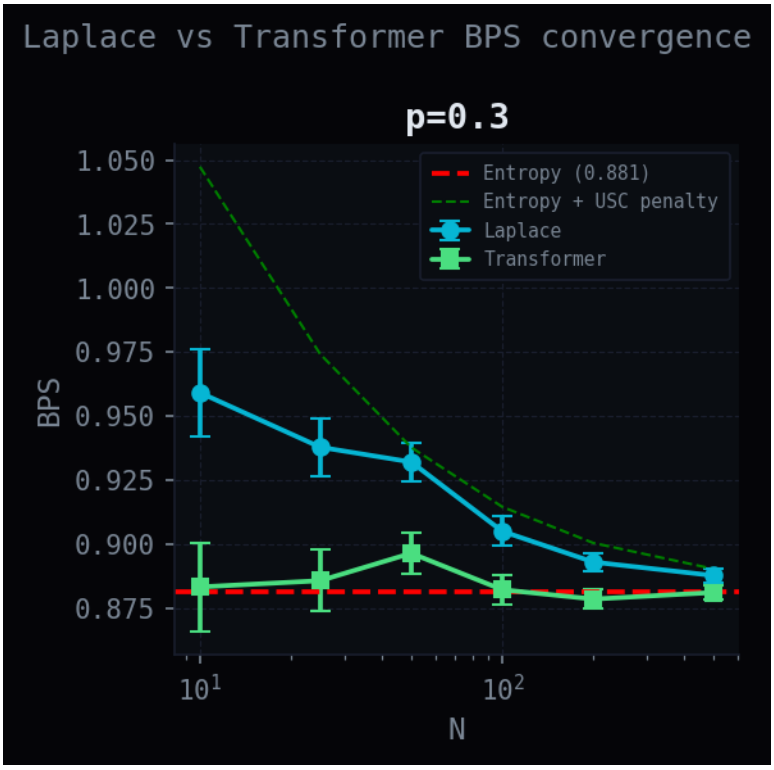
This makes sense. When p is drawn fresh every sequence, there's no cross-sequence structure to exploit. Both models are starting from zero information about the current p. The transformer has learned the Laplace behavior from training, which is the correct thing to do in this setting the best you can do without knowing p is sequential updating from a uniform prior, and that's what both models are effectively doing.

This isn't a failure. There's nothing to beat Laplace with here.

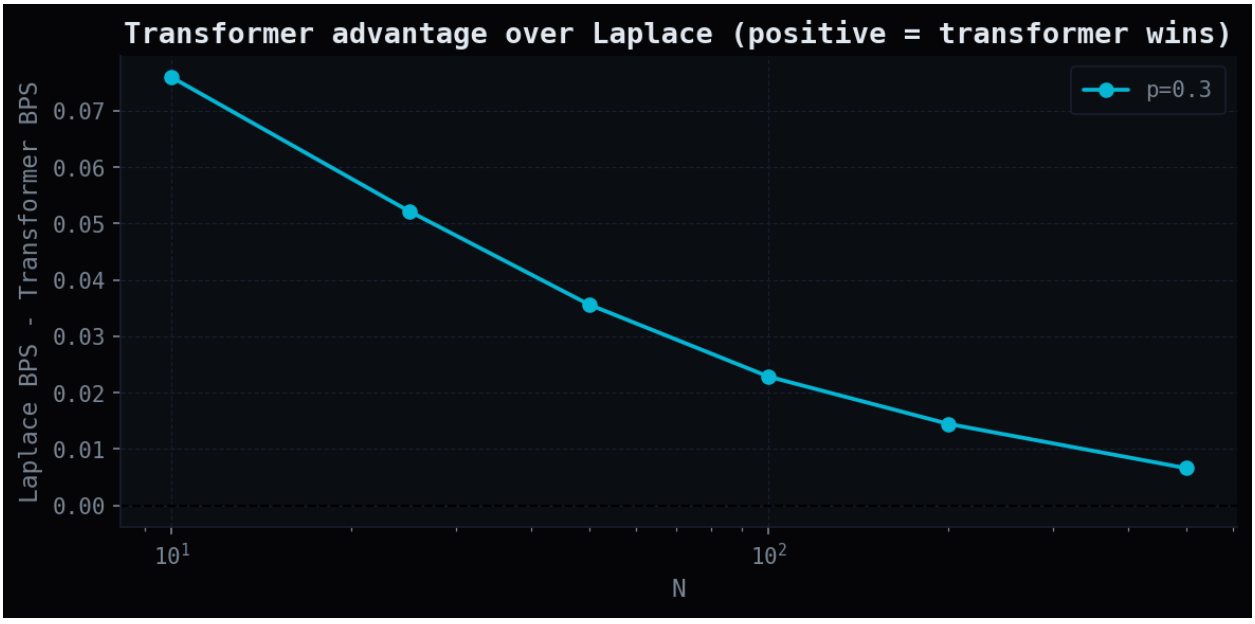
4 Fixed p – Offline Learning Helps

Now the fixed-p case. The transformer is trained on a single p and tested on sequences from that same p.

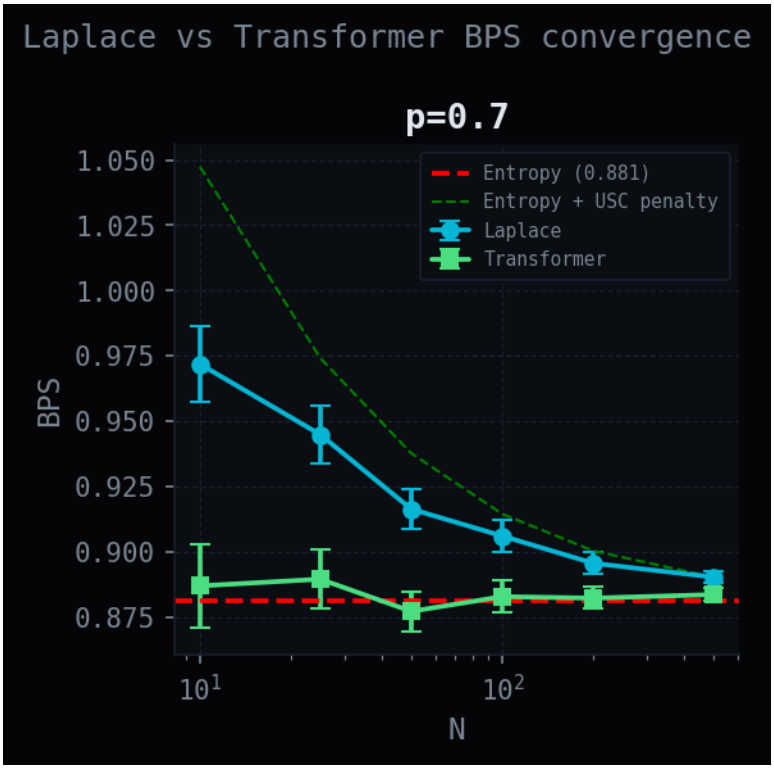
```
# BPS vs N for fixed p=0.3: Transformer (trained offline) starts near entropy from bit one,
```



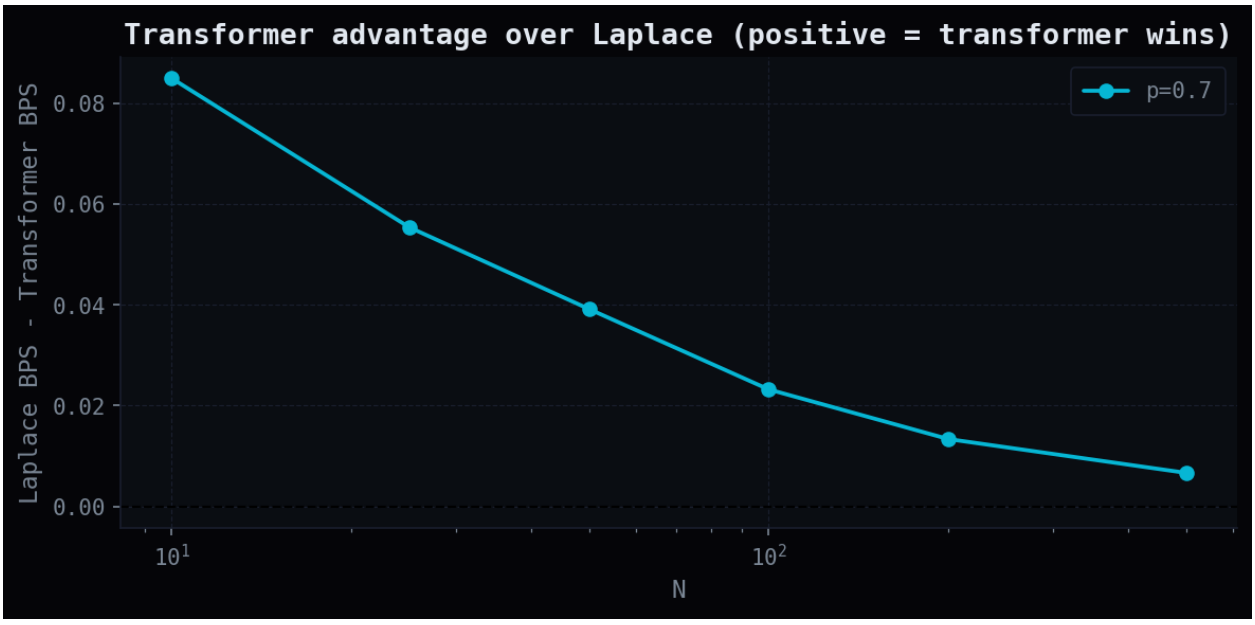

```
# Gap (Laplace - Transformer BPS) vs N for fixed p=0.3.
```




```
# BPS vs N for fixed p=0.7: mirrors the p=0.3 result
```




```
# Gap (Laplace - Transformer BPS) vs N for fixed p=0.7.
```



Here the results are different. For $p = 0.3$, the transformer starts at around 0.883 BPS at $N = 10$, which is essentially at the entropy lower bound of 0.881. Laplace at $N = 10$ is at 0.959 – still paying its estimation cost. The gap at $N = 10$ is about 0.076 bits/symbol and decays cleanly as N grows, converging near zero by $N = 500$ as Laplace accumulates enough counts to catch up.

Same story for $p = 0.7$. The gap is comparable in magnitude and decays the same way.

The mechanism is straightforward. Laplace's redundancy at small N is its estimation cost – it's spending bits learning where p is from the current sequence. The transformer paid that cost during training. At inference time it already knows p, so it starts near entropy immediately. The advantage is real but finite: as N grows, Laplace's running estimate converges and the gap disappears.

5 What This Connects To

The fixed-p condition is the idealized version of a learned coder: infinite training on one distribution, so the learned coder skips all estimation overhead entirely. The gap we observe is exactly Laplace's warm-up cost, avoided by offline learning.

The IID condition shows the other face: when the distribution shifts every sequence, offline training on the marginal distribution of p values doesn't help. The model correctly defaults to universal coding behavior.